

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»

**институт информационных технологий и технологического образования
кафедра информационных технологий и электронного обучения**

Основная профессиональная образовательная программа
Направление подготовки 09.03.01 Информатика и вычислительная техника
Направленность (профиль) «Технологии разработки программного обеспечения»
форма обучения – очная

Курсовая работа

по дисциплине «Технологии компьютерного моделирования»

«Моделирование Фракталов»

Обучающегося (или -ся) 2 курса
ФИО в род.падеже

Салимова Даниила Рамильевича

Руководитель:
к.п.н.доцент
Гончарова С.В.

«_____» _____ 2024 г.

Санкт-Петербург
2024

Оглавление

ВВЕДЕНИЕ.....	3
ТЕОРЕТИЧЕСКАЯ ЧАСТЬ.....	5
1.1. Классификация фракталов.....	6
1.2. Математические основы моделирования фракталов.....	7
ПРАКТИЧЕСКАЯ ЧАСТЬ.....	12
2.1. Инструментарий для моделирования фракталов.....	12
2.2. Построение и анализ фрактальных структур.....	14
ЗАКЛЮЧЕНИЕ.....	20
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ.....	21

ВВЕДЕНИЕ

Актуальность

Что общего у дерева, берега моря, облака или кровеносных сосудов у нас в руке? На первый взгляд может показаться, что все эти объекты ничто не объединяет. Однако на самом деле существует одно свойство структуры, присущее всем перечисленным предметам: они самоподобны. От ветки, как и от ствола дерева, отходят отростки поменьше, от них — еще меньшие, и т. д., то есть ветка подобна всему дереву. В этом они похожи на одни из самых красивых математических объектов — фракталы.

Фракталы являются одними из самых удивительных и многогранных объектов в математике. Их уникальные свойства, такие как самоподобие и дробная размерность, находят широкое применение в различных областях науки и техники. Фракталы можно обнаружить в природе, например, в формах гор, облаков, растений и береговых линий. Кроме того, они широко используются в компьютерной графике для создания сложных и реалистичных изображений, а также в моделировании динамических систем и анализе финансовых рынков. Актуальность изучения фракталов обусловлена их способностью моделировать сложные системы и процессы, что делает их важным инструментом для современных исследований и технологий.

Цель и задачи

Цель данной курсовой работы заключается в изучении методов моделирования фракталов. Для достижения этой цели были поставлены следующие задачи:

1. Исследовать свойства фракталов и их математические основы.
2. Рассмотреть типы фракталов.
3. Проанализировать различные методы моделирования фракталов.
4. Реализовать модели фракталов с помощью языка программирования Python.

Объект исследования: фрактальные структуры и их свойства.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Фрактал (лат. fractus — дроблёный, сломанный, разбитый) — математическое множество, обладающее свойством самоподобия (объект, в точности или приближённо совпадающий с частью себя самого, то есть целое имеет ту же форму, что и одна или более частей). В математике под фракталами понимают множества точек в евклидовом пространстве, имеющие дробную метрическую размерность (в смысле Минковского или Хаусдорфа), либо метрическую размерность, отличную от топологической, поэтому их следует отличать от прочих геометрических фигур, ограниченных конечным числом звеньев. Самоподобные фигуры, повторяющиеся конечное число раз, называются предфракталами.

Фракталы обладают уникальными свойствами, такими как дробная размерность и самоподобие, которые делают их применимыми в различных областях науки и техники.

Самоподобие означает, что фракталы могут быть разбиты на части, каждая из которых является уменьшенной копией всего фрактала. «Системы состоят из подсистем, подсистемы — из подсистем и так до бесконечности — именно поэтому мы проектируем снизу вверх» © Алан Перлис (1922–1990) — американский учёный в области информатики.

1.1. Классификация фракталов

Фракталы можно классифицировать на несколько типов:

1. Геометрические фракталы:

- Примеры включают кривую Коха, треугольник Серпинского и ковер Серпинского. Эти фракталы создаются с помощью простых геометрических правил и обладают высоким уровнем самоподобия. Эти фигуры основаны на прямых линиях, квадратах, кругах, многоугольниках и многогранниках

2. Алгебраические фракталы:

- К таким фракталам относятся множества Мандельброта и Жюлиа. Они определяются алгебраическими уравнениями и демонстрируют сложные, детализированные структуры при увеличении. Алгебраические фракталы, в отличие от геометрических, основываются на формуле, а не на фигурах, но также рекурсивно итерируются.

3. Статистические фракталы:

- Эти фракталы используются для моделирования природных объектов, таких как береговые линии, облака или горные цепи. Они не обладают точным самоподобием, но имеют статистические свойства, которые повторяются на разных масштабах. То есть в отличие от алгебраических и геометрических, формула в этих фракталах не постоянная.

1.2. Математические основы моделирования фракталов

Понятие компьютерной графики не ограничивается такими ее видами как растровая и векторная. В некоторых учебных пособиях по информатике рассматривается третий вид компьютерной графики – фрактальная.

Термином «фрактал» (от латинского fractus – дробленый, состоящий из фрагментов) обозначают геометрическую фигуру, обладающую свойством самоподобия, то есть составленную из нескольких частей, каждая из которых подобна всей фигуре целиком. Этот термин был предложен французским математиком Бенуа Мандельбротом в 1975 году и получил распространение с выходом в 1977 году его книги «Фрактальная геометрия природы».

В настоящее время возрастает интерес к применению методов фрактальной геометрии в различных областях науки и техники – физике, астрономии, биологии, медицине, социологии, экономике, информационных и телекоммуникационных технологиях и др. Так, в физике фракталы применяются при изучении поглощения или рассеяния излучения в пористых средах, при анализе процессов усталостного разрушения материалов, для характеристики сильно развитой турбулентности, при моделировании свойств поверхности твердых тел, для описания диэлектрического пробоя и молнии и пр. В биологии фракталы применяются, например, для моделирования процессов, происходящих во внутренних органах (процессов кровообращения, биения сердца и др.), моделирования популяций. Фракталы используются и для моделирования поведения хаотических динамических систем, таких, как поведение погоды, описан способ моделирования исторических процессов и явлений средствами фрактальной геометрии. В экономике фракталы применяют при анализе колебаний курса валют.

Широко применяются фракталы и в области компьютерной графики, например, для получения изображений природных объектов (растений, облаков, гор, береговых линий, ландшафтов, поверхности морей, карт и пр.) при создании

компьютерных игр. Другое перспективное направление – фрактальное сжатие изображений, позволяющее добиться сверхвысокой степени сжатия (по некоторым данным – до 2000 раз) с малыми потерями качества.

Фрактальная графика, как векторная, является вычисляемой. Но, в отличие от векторной графики, основное внимание уделяется не заданию изображения в виде совокупности простых геометрических фигур (прямых, многоугольников, окружностей, многогранников и пр.), а описанию алгоритма построения изображения – фрактальное изображение строится по уравнению или системе уравнений, и никакие объекты в памяти компьютера не хранятся.

Фракталы часто строятся с помощью итерационных процессов, где одна и та же операция повторяется многократно, в том числе с помощью рекурсии.

Геометрические фракталы получаются путем несложных геометрических построений:

1. задается начальное условие (нулевое поколение): фигура, на основании которой строится фрактал;
2. задается процедура (генератор), которая определенным образом преобразует нулевое поколение;
3. в результате бесконечного повторения заданной процедуры получается геометрический фрактал.

Рассмотрим несколько примеров:

Кривая Коха:

Кривая Коха является типичным геометрическим фракталом. Процесс её построения выглядит следующим образом: берём единичный отрезок, разделяем на три равные части и заменяем средний интервал равносторонним треугольником без этого сегмента. В результате образуется ломаная, состоящая из четырёх звеньев длины $1/3$. На следующем шаге повторяем операцию для каждого из четырёх получившихся звеньев и т. д... Предельная кривая и есть кривая Коха.

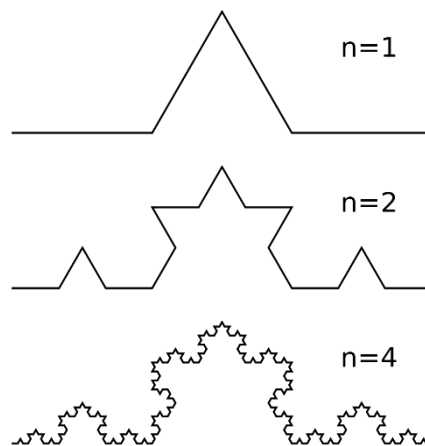


Рисунок 1.

Салфетка Серпинского:

Алгоритм построения салфетки Серпинского следующий:

1. задаем начальное условие: нулевое поколение представляет собой правильный треугольник со стороной 1;
2. определяем процедуру, которая преобразует нулевое поколение: делим средними линиями исходный треугольник на четыре равных треугольника и центральный выбрасываем. Получаем три треугольника со стороной $1/2$;

3. повторяем эту процедуру с тремя оставшимися треугольниками требуемое количество раз.

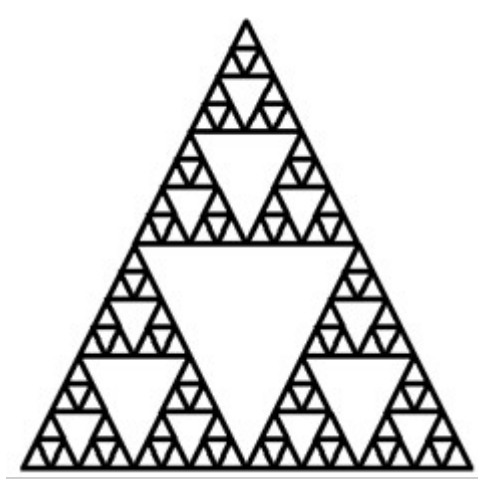


Рисунок 2.

Вторая группа фракталов – алгебраические фракталы. Свое название они получили потому, что их строят на основе алгебраических формул.

Классический пример алгебраических фракталов – множество Мандельброта, описанное французским математиком Пьером Фату еще в 1905 году. Однако впервые оно было построено Бенуа Мандельбротом в 1980 г.

Множество Мандельброта:

При построении множества Мандельброта наиболее часто используется функция комплексного переменного:

$$z_{i+1} = z_i^2 + c,$$

где Z и C – комплексные числа, а $Z_0 = 0$.

Раскладывая уравнение на действительную и мнимую часть, эту формулу заменяют двумя выражениями:

$$\begin{aligned}x_{i+1} &= x_i^2 - y_i^2 + a \\ y_{i+1} &= 2 \cdot x_i \cdot y_i + b\end{aligned},$$

где X_0 и Y_0 для каждой новой точки изначально равны нулю.

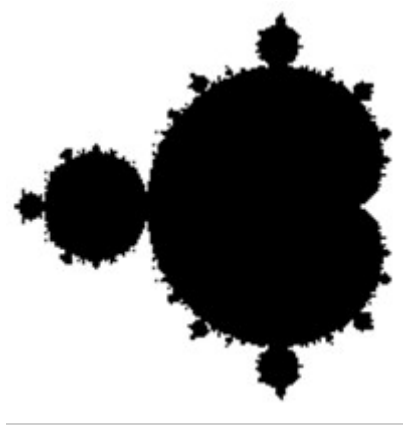


Рисунок 3.

ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1. Инструментарий для моделирования фракталов

Существует множество инструментов и сервисов для моделирования фракталов, каждый из которых предлагает свои уникальные возможности и методы. Среди них есть специальные программы, заточенные под моделирование фракталов, такие как Fractint, Ultra Fractal, помимо их моделирования, они способны еще и на анимацию и цветовую коррекцию фракталов (рисунок 4).

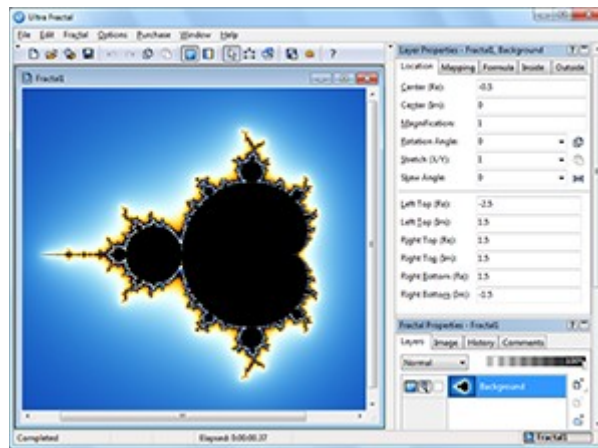


Рисунок 4.

Существуют онлайн-сервисы, такие как FractalLab (рисунок 5), в них менее гибкая настройка моделирования фракталов, но сервис крайне хорош для ознакомления с основными фракталами и их построением.

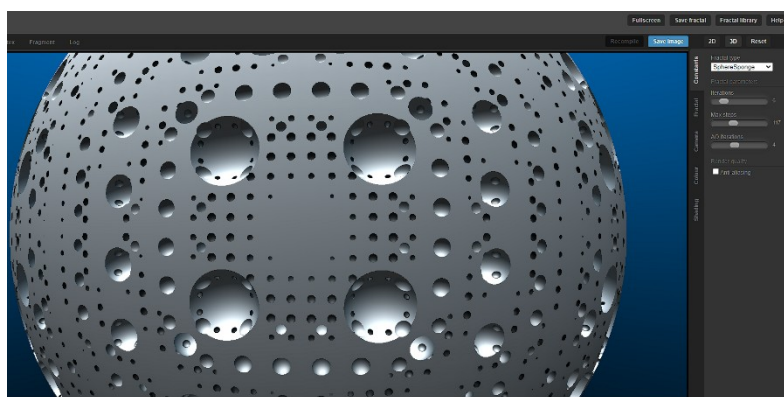


Рисунок 5.

В качестве сервиса для построения фракталов в данной курсовой работе, я хочу показать использования библиотек Turtle, NumPy и Matplotlib на языке Python.

Библиотека turtle — графическая библиотека, встроенная в язык программирования Python. Она предназначена для создания простых, но эффективных графических приложений, основанных на использовании графического интерфейса черепахи (turtle graphics). Он позволяет пользователям рисовать на экране с помощью пикселей, используя набор команд для перемещения черепашки (turtle) и изменения её параметров. Модуль turtle основан на библиотеке tkinter и поставляется вместе со стандартной библиотекой Python.

Matplotlib - библиотека на языке программирования Python для визуализации данных двумерной и трёхмерной графикой. Получаемые изображения могут быть использованы в качестве иллюстраций в публикациях. Matplotlib является гибким, легко конфигурируемым пакетом, который вместе с NumPy, SciPy и IPython предоставляет возможности, подобные MATLAB. В настоящее время пакет работает с несколькими графическими библиотеками.

NumPy: Математические алгоритмы, реализованные на интерпретируемых языках (например, Python), часто работают гораздо медленнее тех же алгоритмов, реализованных на компилируемых языках (например, Фортран, Си, Java). Библиотека NumPy предоставляет реализации вычислительных алгоритмов (в виде функций и операторов), оптимизированные для работы с многомерными массивами. В результате любой алгоритм, который может быть выражен в виде последовательности операций над массивами (матрицами) и реализованный с использованием NumPy, работает так же быстро, как эквивалентный код, выполняемый в MATLAB.

2.2. Построение и анализ фрактальных структур

Используя алгоритм, приведенный в теоретической части для построения геометрических фракталов, реализуем его на языке Python, с его библиотеками Matplotlib и NumPy.

Для начала импортируем все библиотеки.

```
import matplotlib.pyplot as plt
import numpy as np
```

Рисунок 6.

Затем приступаем к написанию функции построения нулевого поколения, то есть фигуры, к которой мы будем в будущем обращаться, в нашем случае – это треугольник.

```
1 usage
def draw_triangle(ax, vertices, color='k'):
    triangle = plt.Polygon(vertices, edgecolor=color, fill=None)
    ax.add_patch(triangle)
```

Рисунок 7.

- ax - объект осей, на которых будет рисоваться треугольник.
- vertices - массив координат вершин треугольника.
- color - цвет границы треугольника (по умолчанию черный).

Теперь определяем рекурсивную функцию “sierpinski”:

```
def sierpinski(ax, vertices, level):
    if level == 0:
        draw_triangle(ax, vertices)
    else:
        # Вычисляем середины сторон текущего треугольника
        midpoints = [
            (vertices[0] + vertices[1]) / 2,
            (vertices[1] + vertices[2]) / 2,
            (vertices[2] + vertices[0]) / 2
        ]

        # Создаем 3 новых треугольника
        sierpinski(ax, vertices=[vertices[0], midpoints[0], midpoints[2]], level=level-1)
        sierpinski(ax, vertices=[vertices[1], midpoints[0], midpoints[1]], level=level-1)
        sierpinski(ax, vertices=[vertices[2], midpoints[1], midpoints[2]], level=level-1)
```

Рисунок 8.

- Функция принимает объект осей ax, вершины треугольника vertices и уровень рекурсии level.
- Если уровень рекурсии равен 0, рисует треугольник.
- Иначе вычисляет середины сторон и рекурсивно вызывает себя для трех новых треугольников с уменьшенным уровнем.

В зависимости от значения “level” будет определена и реализована глубина рекурсии (количество вложений).

Эта часть кода была основной для математического моделирования. Далее мы определяем функцию для визуализации фрактала.

```
1 usage
def plot_sierpinski(level):
    fig, ax = plt.subplots()
    ax.set_aspect('equal')
    ax.axis('off')

    # Начальный большой треугольник
    initial_vertices = np.array([[0, 0], [1, 0], [0.5, np.sqrt(3) / 2]])
    sierpinski(ax, initial_vertices, level)

    plt.show()
```

Рисунок 9.

Эта функция реализует следующие вещи:

- Создает фигуру и оси для рисования.
- Устанавливает равное соотношение осей и отключает оси координат.
- Задает начальные вершины большого треугольника.
- Запускает рекурсивную функцию `sierpinski`.
- Показывает построенное изображение.

В итоге мы получаем фрактал, называемый салфеткой Серпинского

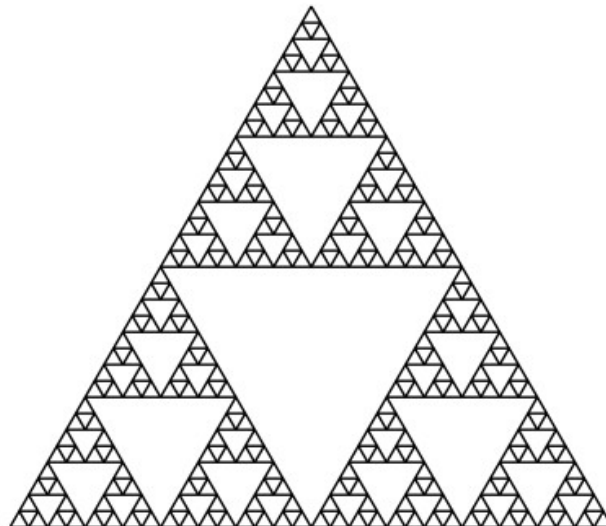


Рисунок 10.

Теперь реализуем Кривую Коха инструментами Python и его библиотекой Turtle.

После импортирования библиотеки, подготавливаем черепаху к рисованию:

```
turtle.hideturtle()
turtle.tracer(0)
turtle.penup()
turtle.setposition(-200, y: 0)
turtle.pendown()
```


Рисунок 11.

- **`turtle.hideturtle()`**: Скрывает курсор черепахи для ускорения рисования.
- **`turtle.tracer(0)`**: Отключает анимацию, чтобы сразу нарисовать конечный результат.
- **`turtle.penup()`**: Поднимает перо, чтобы переместить черепаху без рисования.
- **`turtle.setposition(-200, 0)`**: Устанавливает начальную позицию черепахи.
- **`turtle.pendown()`**: Опускает перо для начала рисования.

Определяем значения и правила логики:

```
axiom = "F"
tempAx = ""
iterable = 4

logic = {
    'F': 'F-F+F-F'
}
```

Рисунок 12.

- **`axiom`**: Начальная строка для L-системы. (в качестве переменной можно использовать любой другой символ)
- **`tempAx`**: Временная строка для хранения промежуточных результатов.
- **`iterable`**: Количество итераций (поколений). (так же как и в предыдущем примере это значение будет влиять на глубину рекурсии)
- **`logic`**: Правила замены символов.

Теперь выполняем итерации для построения строки:

```
for i in range(iterable):
    for j in axiom:
        if j in logic:
            tempAx += logic[j]
        else:
            tempAx += j
    axiom, tempAx = tempAx, ''
```

Рисунок 13.

Проходим через каждое поколение и применяем правила замены для каждого символа в строке axiom.

Теперь рисуем фигуру на основе конечной строки:

```
for k in axiom:
    if k == '+':
        turtle.right(120)
    elif k == '-':
        turtle.left(60)
    else:
        turtle.forward(5)
```

Рисунок 14.

В результате этого цикла мы будем рисовать ломаную из четырех сегментов одинаковой длины



Рисунок 15.

После команд обновления экрана и запуска основного цикла событий для окна черепахи (рисунок 16), мы получаем готовую Кривую Коха (рисунок 17).

```
turtle.update()  
turtle.mainloop()
```

Рисунок 16.

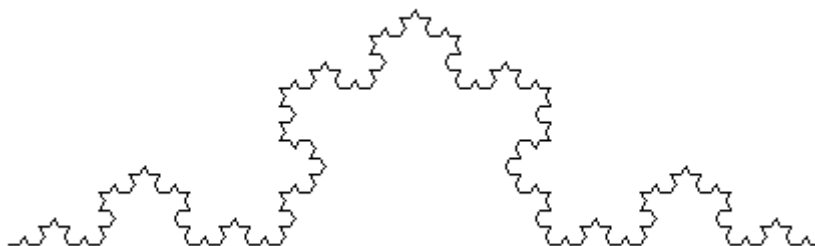


Рисунок 17.

ЗАКЛЮЧЕНИЕ

В практической части курсовой работы были рассмотрены и реализованы различные методы моделирования фракталов с использованием языка программирования Python. Мы изучили метод итерационных функций (IFS), метод Линденмайера (L-системы). примеры кода и визуализации фракталов наглядно продемонстрировали, как можно создавать сложные фрактальные структуры с помощью относительно простых алгоритмов и инструментов. Эти методы моделирования позволяют исследовать уникальные свойства фракталов и их практическое применение в компьютерной графике, природных моделях и финансовом анализе.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Бенуа Мандельброт, "Фрактальная геометрия природы" (Benoit B. Mandelbrot, "The Fractal Geometry of Nature")
 - Классическая работа, в которой Мандельброт вводит понятие фракталов и описывает их свойства и применение в природе.
2. Майкл Барнсли, "Фракталы повсюду" (Michael F. Barnsley, "Fractals Everywhere")
 - Книга предлагает глубокое понимание фракталов и различных методов их генерации, включая IFS.
3. Хайнц-Ото Пейтген, Дитер Сауве, "Фракталы: от живописи до компьютеров" (Heinz-Otto Peitgen, Dietmar Saupe, "The Science of Fractal Images")
 - Исследует применение фракталов в искусстве и компьютерной графике, а также методы их построения.
4. Юрий Е. Несмеянов, "Фракталы и их применение в науке и технике"
 - Обзор применения фракталов в различных областях науки и техники с акцентом на практические аспекты.

ПРИЛОЖЕНИЕ А

Моделирование Салфетки Серпинского

```
import matplotlib.pyplot as plt
import numpy as np

def draw_triangle(ax, vertices, color='k'):
    triangle = plt.Polygon(vertices, edgecolor=color, fill=None)
    ax.add_patch(triangle)

def sierpinski(ax, vertices, level):
    if level == 0:
        draw_triangle(ax, vertices)
    else:
        # Вычисляем середины сторон текущего треугольника
        midpoints = [
            (vertices[0] + vertices[1]) / 2,
            (vertices[1] + vertices[2]) / 2,
            (vertices[2] + vertices[0]) / 2
        ]

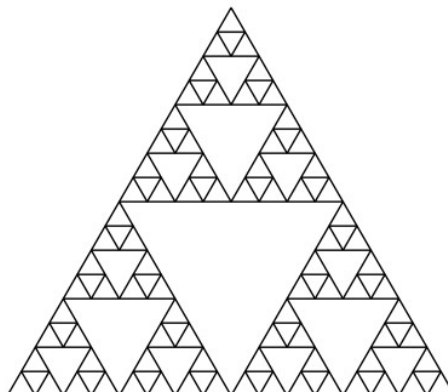
        # Создаем 3 новых треугольника
        sierpinski(ax, [vertices[0], midpoints[0], midpoints[2]], level - 1)
        sierpinski(ax, [vertices[1], midpoints[0], midpoints[1]], level - 1)
        sierpinski(ax, [vertices[2], midpoints[1], midpoints[2]], level - 1)

def plot_sierpinski(level):
    fig, ax = plt.subplots()
    ax.set_aspect('equal')
    ax.axis('off')

    # Начальный большой треугольник
    initial_vertices = np.array([[0, 0], [1, 0], [0.5, np.sqrt(3) / 2]])
    sierpinski(ax, initial_vertices, level)

    plt.show()

# Укажите количество уровней рекурсии
plot_sierpinski(level=4)
```



ПРИЛОЖЕНИЕ Б

Моделирование Кривая Коха

```
import turtle

turtle.hideturtle()
turtle.tracer(0)
turtle.penup()
turtle.setposition(-200, 0)
turtle.pendown()

axiom = "F"
tempAx = ""
iterable = 4

logic = {
    'F': 'F-F+F-F'
}

for i in range(iterable):
    for j in axiom:
        if j in logic:
            tempAx += logic[j]
        else:
            tempAx += j
    axiom, tempAx = tempAx, ''

for k in axiom:
    if k == '+':
        turtle.right(120)
    elif k == '-':
        turtle.left(60)
    else:
        turtle.forward(5)

turtle.update()
turtle.mainloop()
```

